Assignment 1 - Question 3: Graph Indexing Heuristic Report

1 Introduction

In this section, we describe the heuristic used for selecting discriminative subgraphs for the Graph Indexing problem. We also detail an experimental Voting Mechanism that was explored to improve the indexing performance.

2 Final Heuristic: gIndex Approach

For the final submission, we implemented a discriminative subgraph selection strategy based on the **gIndex** methodology, as proposed by Yan et al. [1]. It is worth noting that our initial baseline method (utilizing simple frequency-based selection) yielded a score of approximately **0.19**. To improve upon this, we adopted the following discriminative pipeline.

2.1 Methodology

The goal is to select a compact set of $K = 50$ features (subgraphs) that maximize pruning power. The selection pipeline operates as follows:

1. **Frequent Subgraph Mining:** We mine frequent subgraphs using the gSpan algorithm with a minimum support threshold of 25%.

2. **Discriminative Pruning (Redundancy Check):** A frequent subgraph g is considered redundant if it does not provide significantly more pruning power than its sub-structures (parents). A graph g is discarded if its support is close to the support of any of its parents g_{parent} (where size(g_{parent}) = size(g) - 1).

$$\text{Discard } g \text{ if } \exists g_{parent} \subset g \text{ such that } support(g) \geq 0.9 \times support(g_{parent}) \tag{1}$$

   This ensures we only retain larger features when they represent specific, frequent interactions that are significantly rarer than their components.

3. **Size Filtering:** While mining, we have put a minimum of 2 and a maximum vertices criteria of 10 to mine meaningful molecular structures.

4. **Ranking by Information Content:** From the remaining discriminative subgraphs, we rank them based on how effectively they split the database. The "ideal" feature divides the database evenly (50% presence). We use the following score:

$$Score(g) = |support(g) - 0.5 \times |DB|| \tag{2}$$

   Features are sorted in **ascending order** of this score (closer to 0 is better), and the top 50 are selected.

2.2 Results

Using the gIndex-based approach described above, we achieved the following scores on the benchmark datasets:

- **Mutagenicity:** 0.2544
- **NCI-H23:** 0.2856

3 Approach

To investigate potential improvements over the standard heuristic suggested by the paper, we devised a **Voting Mechanism** based on the ground truth answer sets $R(q)$. This experiment aimed to identify features that are most effective at pruning false positives for the actual query workload so that Top 50 graphs can be selected.

3.1 Mechanism Details

3.1.1 Candidate Pool Generation

We first generated a larger pool of potential features by mining discriminative subgraphs with a lower minimum support threshold of **25%**.

3.1.2 Voting System

We implemented a voting system where each query graph q in the dataset casts votes for features that help prune its specific false positives. For every query graph q, we score every discriminative graph d in the pool as follows:

1. We iterate over every graph g in the database.
2. **Case 1 (True Positive):** If $g \in R(q)$ (i.e., g is a supergraph of q), this is a correct answer. We represent this as adding 0 to the score of d.
3. **Case 2 (False Positive Candidate):** If $g \notin R(q)$, we check if feature d can prune g:
   - **Pruning Success:** If $d \subseteq q$ AND dg, then feature d is present in the query but missing in the database graph g. This allows us to prune g safely. **We add 1 to the score of d.**
   - **Pruning Failure:** Otherwise, the feature fails to prune g (either it's missing in q or present in g). **We subtract 1 from the score of d.**

After iterating through the database, each feature d has a net score for query q. The query q then casts **one vote** to each of the **Top 50** highest-scoring discriminative graphs.

3.1.3 Final Selection

We aggregate the votes from all queries. The final set of 50 features is selected by taking the graphs with the maximum number of votes. In case of ties, we select the graphs with support closer to 50%.

3.2 Conclusions

This voting-based selection yielded significantly higher scores than the purely heuristic approach:

- **Mutagenicity:** Score improved from ≈ 0.25 to ≈ 0.3.
- **NCI-H23:** Score improved from ≈ 0.28 to ≈ 0.4.

We analyzed the size and support distribution of the "Top 50 Voted Graphs" (hypothetically optimal features) against the general population of discriminative graphs. Through this analysis, we found that no heuristic based purely on graph size was effective. However, we observed that the optimal graphs consistently exhibited support levels centered around the median of the database. Consequently, the heuristic $|support(g) - 0.5 \times |DB||$, inspired by how entropy is calculated in information theory (where uncertainty/information is maximized when probabilities are equal), was found to be the most helpful metric to select the top 50 graphs.

References

[1] Xifeng Yan, Philip S. Yu, and Jiawei Han. "Graph Indexing: A Frequent Structure-based Approach". In *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data*, 2004.

[2] Internal Log. "Benchmarking gIndex Pipeline.md". 2026.

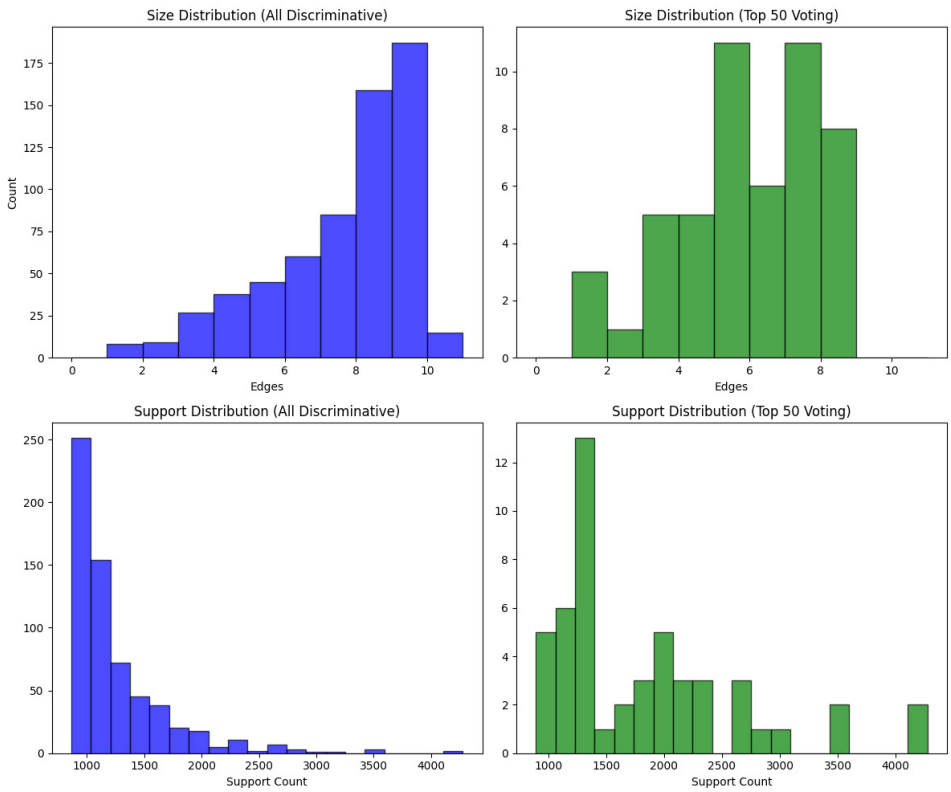

Figure 1: Comparison of support and size distribution between the optimal Top 50 voted graphs and the entire discriminative subgraph set.

[3] Internal Log. "Implementing Baseline Identification.md". 2026.

[4] Internal Log. "Reverting Voting Mechanism.md". 2026.

[5] Internal Log. "Confirming Final Heuristic.md". 2026.

A Development Logs

A.1 Benchmarking gIndex Pipeline

```
# Chat Conversation

Note: _This is purely the output of the chat conversation and does not contain any raw data, codebas

### User Input

calculate score for ncl dataset, the required files are already generated

*Listed directory [q3](file:///home/arnav/Sixth%20semester/COL%20761/BitBlasting/A1/q3) *

*Viewed [score_calculator.py](file:///home/arnav/Sixth%20semester/COL%20761/BitBlasting/A1/q3/score_

*Edited relevant file*

*User accepted the command `python3 score_calculator.py nci_candidates.txt nci_rq.pkl`*

*Edited relevant file*

### Planner Response
```

```
Calculated the score for the NCI dataset.

**Average Score:** ‘0.2234‘

**Output Summary:**
- **Queries processed**: 50
- **Sample Scores**:
    - Query 0: 0.2166
    - Query 5: 0.4788 (Highest in first 10)
    - Query 8: 0.0095 (Lowest in first 10)

*Edited relevant file*

*User accepted the command ‘git push‘*
...
```

A.2 Implementing Baseline Identification

```
# Chat Conversation

Note: _This is purely the output of the chat conversation and does not contain any raw data, codebas

### User Input

read the assignment pdf for q3, the implementation of q3 is already done

now i want to create a score calculating mechanism, for which i need to know the Rq for every query

find Rq for every graph and the design the score calculator

*Searched filesystem*

*Edited relevant file*

*Read URL content from /home/arnav/Sixth semester/COL 761/BitBlasting/A1/2502_COL761_A1_final.pdf *
...
```

A.3 Reverting Voting Mechanism

```
# Chat Conversation

Note: _This is purely the output of the chat conversation and does not contain any raw data, codebas

### User Input

generate the discriminative graphs with minsup 20%

then define a voting system:
each query graph q, scores every discriminative graphs d as follows:
iterate over every graph g in the database
case 1: if g belongs to rq
add 0 to the score

case2: g doesnot belong to cq
1. if d belongs to q but doesnot belong to g, add 1
2. otherwise subtract 1
```

```
each query graph casts one vote to the graphs with top 50 scores

at the end, i select 50 graphs with max number of votes, in case of ties, select graphs with support

implement this strategy and plot size distribution and frequency distribution of the top 50 discrimi
...
```

A.4 Confirming Final Heuristic

```
# Chat Conversation

Note: _This is purely the output of the chat conversation and does not contain any raw data, codebas

### User Input

do you know the final heuristic for q3 of this assignment

*Viewed [identify_gindex.py](file:///home/arnav/Sixth%20semester/COL%20761/BitBlasting/A1/q3/identif

*Viewed [run_code.sh](file:///home/arnav/Sixth%20semester/COL%20761/BitBlasting/A1/run_code.sh) *

### Planner Response

Based on the codebase and our recent work ensuring the correct implementation, the **final heuristic

The heuristic selects the most "informative" subgraphs to act as index features by following these s

1.  **Discriminative Pruning (Redundancy Check)**:
    A subgraph $g$ is discarded as "redundant" (not discriminative) if its support is too close to t
...
```